

NeuralChain AI Model Marketplace

A compact guide to the connections, data flow, technologies, and decentralization trade-offs

In one sentence: React is the user interface, Express is the application/API layer, MongoDB stores searchable marketplace records, Pinata/IPFS stores model files by content address, and Solidity contracts enforce listings, payments, access, royalties, governance tokens, and license NFTs on an EVM blockchain.

1. System Map

```
Browser (React + Vite + ethers.js + MetaMask) | REST/JSON | signed blockchain transactions v v
Express API (Node.js) ---- RPC ----> EVM chain / Hardhat / Sepolia | \ | ModelMarketplace | \ |
ModelNFT + NeuralToken | v v MongoDB Atlas Pinata API ---->
      IPFS gateway / content-addressed model file (catalog, users, (optional) (CID / modelHash
in the listing) purchases, reviews)
```

The chain is the trust and settlement layer. The backend is the coordination, authentication, moderation, search, download authorization, and verification layer. IPFS is the large-file layer; MongoDB is a fast application index and history store.

2. What Connects to What

Part	Connection and responsibility	Language / advantage
Frontend	Calls <code>/api/*</code> with Axios and asks MetaMask/ethers.js to sign listings and purchases.	React, JavaScript, CSS. Fast interactive marketplace UI and wallet-native signing.
Backend	Express routes connect auth, model listings, IPFS upload, purchases, governance, reviews, and admin moderation.	Node.js / JavaScript. Central place for validation, rate limits, JWTs, orchestration, and APIs.
MongoDB	Mongoose models persist users, model metadata, owners, hashes, contract IDs, purchases, reviews, and versions.	JavaScript documents and indexes. Flexible metadata plus efficient search and dashboards.
Pinata/IPFS	The backend sends a verified model buffer to Pinata. Pinata returns an IPFS CID; the CID is saved with the model and placed in the contract listing.	HTTP API plus IPFS. Content addressing makes changed content produce a different identifier.
RPC provider	The frontend submits signed transactions to the network; the backend reads receipts and logs through <code>RPC_URL</code> .	JSON-RPC + ethers.js. Shared, independently verifiable chain state.
Smart contracts	ModelMarketplace records listings and settles ETH/NEURAL purchases; it calls <code>ModelNFT</code> to mint a license.	Solidity 0.8.x. Deterministic rules that do not depend on one server being honest.

3. Upload and Listing Flow

1. The creator selects a model in React. The browser uploads it to `POST /api/ipfs/upload`.
2. Multer keeps the upload in memory and limits it to 100 MB. Static verification checks extension, size, file signature, archive paths, suspicious commands, and calculates SHA-256.
3. With valid `PINATA_JWT` or API keys, Express calls Pinata's `pinFileToIPFS` endpoint. Pinata returns a CID. Without credentials, the demo creates a deterministic CID from the SHA-256 multihash; this is not durable pinning.

4. The frontend submits listing metadata and the creator signs `uploadModelWithLineage`. The contract requires a non-empty IPFS hash, a pending moderation status, and a minimum price.
5. The backend stores the searchable copy in MongoDB and also maintains JSON fallback data for resilient demos. The contract ID and transaction hash link the application record to the chain.
6. An admin can approve the listing. Public marketplace queries expose only verified or legacy records.

4. Contracts and Transaction Workings

ModelMarketplace.sol

- Stores model owner, name, price, IPFS CID, model hash, verification status, score, and optional parent model.
- Accepts ETH or NEURAL payments.
- Uses personal, commercial, and enterprise multipliers of 1x, 3x, and 10x.
- Splits revenue: 10% protocol fee; normally 90% creator share; fine-tunes also pay a 10% parent-lineage royalty.
- Emits events used by the backend for receipt verification.

ModelNFT.sol

- ERC-1155 license token minted after purchase.
- Ownership is transferred to the marketplace during deployment, allowing the marketplace to mint.
- Transfers are blocked except minting and burning, so a license represents wallet-specific access rather than a tradable collectible.

NeuralToken.sol

- ERC-20 token named NEURAL with permit and vote support.
- Used for purchases and governance-related voting power.
- The deployment script initially mints the fixed maximum supply and funds a local demo buyer.

Receipt verification

- Backend loads the marketplace ABI and queries the transaction plus receipt through the RPC.
- It parses `ModelPurchased` or `NeuralPurchase`, then records model ID, tier, amount, and block number.
- Production must use a live RPC and real contract address; local fallback is explicitly demo behavior.

5. Purchase and Access Flow

```
Buyer selects tier -> MetaMask estimates/signs transaction -> contract checks model, price, tier,
and payment -> funds split to creator / parent creator / platform -> license NFT minted -> receipt
event read by backend -> MongoDB purchase recorded
-> download or inference access allowed
```

ETH purchases use the transaction value. NEURAL purchases require an ERC-20 allowance and use `transferFrom`. The contract prevents a buyer from purchasing an equal or lower tier twice. The API then links the wallet, transaction hash, contract model ID, and license tier to an application purchase.

6. MongoDB, Authentication, and Supporting Services

- **MongoDB Atlas:** stores records that are awkward or expensive to query on-chain: profiles, descriptions, tags, benchmarks, moderation notes, reviews, downloads, API keys, and purchase history. Mongoose adds schemas and indexes.
- **JWT authentication:** Express issues tokens for registered users; Axios attaches the token to API calls. Wallet addresses are also used for Web3 identity and access checks.
- **Moderation:** static file verification runs before upload completion, while admin review controls marketplace visibility. The verifier does not execute uploaded model code or measure real accuracy.
- **API gateway:** developer keys support model inference-style endpoints and rate-limited access after purchase.
- **Deployment:** Hardhat compiles/deploys contracts; the deployment script synchronizes ABI and addresses to frontend and backend. Express can serve the built React app as one production service.

7. Decentralized vs Centralized

Concern	NeuralChain approach	Typical centralized marketplace
Ownership and payment	Contract state, wallet signatures, event logs, and programmable revenue splits.	Company database and private payment ledger.
Artifact integrity	IPFS CID and model hash reveal whether a downloaded artifact changed.	Provider controls storage and can replace files unless extra audit controls exist.
Portability	Wallet-based identity and a public EVM contract can be inspected by other clients.	Account, balance, and access usually stay inside one platform.
Creator economics	On-chain fee and lineage rules execute automatically and are publicly auditable.	Fees and royalties depend on the platform's internal enforcement.
Availability	Chain state is replicated; IPFS can be served by multiple gateways when content is pinned.	Outage or account suspension can remove access to the service.
Trade-offs	Gas fees, confirmation time, wallet UX, irreversible mistakes, RPC dependence, and smart-contract risk.	Faster UX, easier refunds, simpler recovery, but greater platform custody and trust.

8. Important Boundary

This project is hybrid, not fully decentralized. MongoDB, Express, Pinata, moderation, JWT authentication, and download authorization are centralized services. The decentralized core is the wallet-signed contract state and payment settlement, plus content-addressed IPFS references. A production deployment should pin content with a durable Pinata plan or multiple pinning providers, use a real testnet/mainnet RPC, remove demo fallbacks, protect secrets, and add an isolated model execution sandbox.

9. Technology Summary

Languages: JavaScript (React frontend and Node.js backend), Solidity (EVM contracts), JSON (ABIs, configuration, fallback data), HTML/CSS (this guide), and shell configuration for deployment.

Core libraries and tools: React, Vite, Axios, ethers.js, Express, Mongoose, Multer, JWT, Hardhat, OpenZeppelin, MetaMask, MongoDB Atlas, Pinata, and IPFS.

Prepared from the current NeuralChain repository implementation. Local/demo fallbacks are called out where they differ from production behavior.